

JavaScript

What is Scripting Language?

- A scripting language is a programming language designed for integrating and communicating with other programming languages.
- Some of the most widely used scripting languages are JavaScript, VBScript, PHP, Perl, Python, Ruby, ASP.
- Scripting language is normally used in combining with another programming language, they are often found alongside HTML, Java or php.

What is JavaScript ?

- JavaScript is a Scripting Language.
- JavaScript is an Open Source Element.
- JavaScript is Platform Independent Scripting Language.
- JavaScript is the Case Sensitive Language.
- JavaScript is developed by the NetScape Corporation.
- JavaScript is Object Base Language.
- JavaScript is the Event-Driven Programming Language.

Where to put JavaScript ?

- In HTML, JavaScript code must be inserted between `<script>` and `</script>` tags.
- You can place any number of scripts in an HTML document.
- Scripts can be placed in the `<body>`, or in the `<head>` section of an HTML page, or in both.
- Placing scripts at the bottom of the `<body>` element improves the display speed, because script compilation slows down the display.
- In **.js** file (external JavaScript)

External JavaScript ?

- Scripts can also be placed in external files.
- JavaScript files have the file extension **.js**.
- External scripts are practical when the same code is used in many different web pages.
- External scripts cannot contain `<script>` tags.
- To use an external script, put the name of the script file in the **src** (source) attribute of a `<script>` tag.
- E.g : `<script type="text/javascript" src="myScript.js">
</script>`

External JavaScript Advantages ?

- When the same code is used in many different web pages.
- It separates HTML and javascript code.
- It makes HTML and JavaScript easier to read and maintain.
- Cached JavaScript files can speed up page loads.

JavaScript Output

- JavaScript can "display" data in different ways:
 - Writing into an HTML element, using **innerHTML**.
 - Writing into the HTML output using **document.write()**.
 - Writing into an alert box, using **window.alert()**.
 - Writing into the browser console, using **console.log()**.

Using innerHTML

- To access an HTML element, JavaScript can use the `document.getElementById(id)` method.
- The `id` attribute defines the HTML element. The `innerHTML` property defines the HTML content:

Using document.write()

- The document.write() method should only be used for testing.

Using console.log()

- For debugging purposes, we can use the console.log() method to display data.

JavaScript Identifier

- All JavaScript variables must be identified with unique names.
- These unique names are called identifiers. Eg: age, Salary.
- The general rules for constructing names for variables (unique identifiers) are :
 - Names can contain letters, digits, underscores, and dollar signs.
 - Names must begin with a letter, \$ and _ .
 - Names are case sensitive.
 - Reserved words (like JavaScript keywords) cannot be used as names

JavaScript Variables

- Variables are Containers for Storing Data
- JavaScript Variables can be declared in 4 ways:
 - Automatically
 - Using var
 - Using let
 - Using const

JavaScript Variables

- **Automatically**
- **Example**
 - `x = 5;`
`y = 6;`
`z = x + y;`
- In this example, x, y, and z are undeclared variables.
- They are automatically declared when first used:

JavaScript Variables

Var

- The var keyword was used in all JavaScript code from 1995 to 2015.
- The var keyword is also used to declare global-scope variables.
- The var keyword should only be used in code written for older browsers.
- Re-declare a JavaScript variable declared with var, will not lose its value

JavaScript Variables

Let

- The let keyword was introduced in ES6 (2015).
- Variables defined with let can not be redeclared.
- Variables defined with let must be declared before use.
- Variables defined with let have block scope.

Const

- The const keyword was introduced in ES6 (2015).
- Variables defined with const cannot be Redeclared
- Variables defined with const cannot be Reassigned
- Variables defined with const have Block Scope.

JavaScript Comments

- JavaScript comments can be used to explain JavaScript code, and to make it more readable.
- JavaScript comments can also be used to prevent execution, when testing alternative code
 - Single line comments start with `//`.
 - Multi-line comments start with `/*` and end with `*/`. It is also called comment block.

JavaScript Operators

- These are symbols that are used to manipulate a certain value or operand. Operators are used to performing specific mathematical and logical computations on operands.
- There are different types of JavaScript operators:
 - Arithmetic Operators
 - Assignment Operators
 - Comparison Operators
 - Logical Operators
 - Bitwise Operators
 - Ternary Operators
 - Type Operators

JavaScript Operators

Arithmetic Operators : Arithmetic Operators are used to perform arithmetic on numbers.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (<u>ES2016</u>)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

JavaScript Operators

Assignment Operators : Assignment operators assign values to JavaScript variables.

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

JavaScript Operators

Comparison Operators : Comparison operators compares values of JavaScript variables. All the comparison operators can also be used on strings.

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

JavaScript Operators

Logical Operators : Logical operators are generally used for combining two or more relational statements.

Operator	Description
&&	logical and
	logical or
!	logical not

JavaScript Operators

Type Operators :

Operator	Description
typeof	Returns the type of a variable
instanceof	Returns true if an object is an instance of an object type

JavaScript Operators

Bitwise Operators : Bit operators work on 32 bits numbers.

Any numeric operand in the operation is converted into a 32 bit number. The result is converted back to a JavaScript number.

Operator	Description	Example	Same as	Result	Decimal
&	AND	5 & 1	0101 & 0001	0001	1
	OR	5 1	0101 0001	0101	5
~	NOT	~ 5	~0101	1010	10
^	XOR	5 ^ 1	0101 ^ 0001	0100	4
<<	left shift	5 << 1	0101 << 1	1010	10
>>	right shift	5 >> 1	0101 >> 1	0010	2
>>>	unsigned right shift	5 >>> 1	0101 >>> 1	0010	2

JavaScript Conditional Statements

- Conditional statements are used to perform different actions based on different conditions.
- if, else, else if and switch

JavaScript Loops

JavaScript supports different kinds of loops:

- **while** - loops through a block of code while a specified condition is true
- **do/while** - also loops through a block of code while a specified condition is true
- **for** - loops through a block of code a number of times
- **for/in** - loops through the properties of an object
- **for/of** - loops through the values of an iterable object

JavaScript Loops

For In Loop : JavaScript for-in loop is used to iterate over the properties of an object.

Syntax : for (key in object) {
 // code block to be executed
}

For Of Loop : The JavaScript for of statement loops through the values of an iterable object.

Syntax : for (variable of iterable) {
 // code block to be executed
}

JavaScript Data Types

- JavaScript variables can hold many data types
 - Boolean
 - Null
 - Undefined
 - Number
 - String
 - Object

JavaScript Data Types

- **Undefined**
 - In JavaScript, a variable without a value, has the value undefined. The typeof is also undefined.
 - Eg: **var age;** // Value is undefined, type is undefined
- Any variable can be emptied, by setting the value to undefined. The type will also be undefined.
 - Eg: **age = undefined;** // Value is undefined, type is undefined

JavaScript Data Types

- **Null**

- In JavaScript null is "nothing". It is supposed to be something that doesn't exist.
- In JavaScript, the data type of null is an object.

Difference Between Undefined and Null

- Undefined and null are equal in value but different in type:
 - `typeof undefined` // undefined
 - `typeof null` // object

NaN

- NaN stands for Not a Number.
- It is a special numeric value that indicates an invalid number.
- For example, the division of a string by a number returns NaN.
- **The NaN has two special characteristics:**
 - Any operation with NaN returns NaN.
 - The NaN does not equal any value, including itself.

The number type

- JavaScript uses the number type to represent both integer and floating-point numbers.
- To represent a floating-point number, you include a decimal point followed by at least one number.
- JavaScript automatically converts a floating-point number into an integer if the number appears to be a whole number.
- Javascript always wants to use less memory since a floating-point value uses twice as much memory as an integer value.

The string type

- In JavaScript, a string is a sequence of zero or more characters. A string literal begins and ends with either a single quote(') or a double quote (").
- A string that begins with a double quote must end with a double quote, and a string that begins with a single quote must also end with a single quote.
- JavaScript strings are immutable, meaning that they cannot be modified once created. However, you can create a new string from an existing one

The boolean type

- The boolean type has two literal values: **true** and **false** in lowercase.
- To convert values of other types into boolean values, you use the **Boolean()** function.
- The following table displays the conversion rules:

Type	true	false
string	non-empty string	empty string
number	non-zero number and Infinity	0, NaN
object	non-null object	null
undefined		undefined

The object type

- In JavaScript, an object is a collection of properties, where each property is defined as a key-value pair.

```
let person = {  
  firstName: 'John',  
  lastName: 'Doe'  
};
```

The bigint type

- The bigint type represents the whole numbers that are larger than $2^{53} - 1$.
- To form a bigint literal number, you append the letter **n** at the end of the number:

JavaScript Functions

- A JavaScript function is a block of code designed to perform a particular task.
- A JavaScript function is executed when "something" invokes it (calls it).
- A JavaScript function is defined with the **function** keyword, followed by a name, followed by parentheses ().
- Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).
- The parentheses may include parameter names separated by commas:(parameter1, parameter2, ...)
- The code to be executed, by the function, is placed inside curly brackets: { }

JavaScript default parameters in ES6.

Default function parameters allow you to initialize named parameters with default values if no values or undefined are passed into the function.

```
function say(message='Hi') {  
    console.log(message);  
}
```

```
say(); // 'Hi'
```

```
say('Hello') // 'Hello'
```

JavaScript functions

- JavaScript treats function as a first-class citizen.
- It treats functions like values of other types.
- It can be stored as a value in a variable.
- It can be returned by another function.
- It can be passed into another function as an Argument.
- It can also stored in an array, queue, or stack.
- It can have its own methods and property.

JavaScript anonymous functions

- An anonymous function is a function without a name.
- **Syntax :**
 `(function () {`
 `//...`
 `});`
 - Anonymous function are put inside the parentheses (),
 - The parentheses () make the anonymous function an expression that returns a function object.
- An anonymous function is not accessible after its initial creation. Therefore, you often need to assign it to a variable.

JavaScript anonymous functions

- An anonymous function is not accessible after its initial creation. Therefore, you often need to assign it to a variable.

```
let show = function() {  
    console.log('Anonymous function');  
};  
  
show();
```

- We need to call the anonymous function later, we assign the anonymous function to the show variable.

Immediately Invoked Function Expressions

- (IIFE) are JavaScript functions that are executed immediately after they are defined.
- The parentheses () make the anonymous function an expression that returns a function object.

```
(function() {  
    console.log('Hello');  
})();
```

- Call the function by adding the trailing parentheses ()

JavaScript Arrow functions

ES6 arrow functions provide you with an alternative way to write a shorter syntax compared to the function expression.

arrow functions with no parameter : If the arrow function has no parameter, we need to use parentheses, like this:

```
() => { statements }
```

the arrow function has one expression $x + y$ so it returns the result of the expression.

```
let add = (x, y) => x + y;  
  
console.log(add(10, 20)); // 30;
```

JavaScript Arrow functions

if block syntax is used, we need to specify the return keyword:

```
let add = (x, y) => { return x + y; };
```

JavaScript rest parameters

ES6 provides a new kind of parameter so-called rest parameter that has a prefix of three dots (...). A rest parameter allows you to represent an indefinite number of arguments as an array.

```
function sum(...args)
{
    let total = 0;
    for (const a of args)
    {
        total += a;
    }
    return total;
}

sum(1, 2, 3);
```

JavaScript String

- A JavaScript string is zero or more characters written inside quotes.
- Strings created with single or double quotes works the same.
- There is no difference between the two.

- **Template Strings :**
- Templates were introduced with ES6 (JavaScript 2016).
- Templates are strings enclosed in backticks.
 - **let text = `He's often called "Mahi"`;**
- Templates allow single and double quotes inside a string:

JavaScript String

Interpolation :

Template String provide an easy way to interpolate variables and expressions into strings.

The method is called string interpolation.

Syntax : \${...}

```
let firstName = "John";  
let lastName = "Doe";  
  
let text = `Welcome ${firstName}, ${lastName}!`;
```

Automatic replacing of variables with real values is called string interpolation.

JavaScript String

- **indexOf()** – get the index of the first occurrence of a substring in a string.
 - **Syntax : `string.indexOf(substr, [, fromIndex])`**
- **lastIndexOf()** – find the index of the last occurrence of a substring in a string.
 - **Syntax : `string.lastIndexOf(substr, [, fromIndex]);`**
- **includes()** – check if a string contains a substring.
 - **Syntax : `string.includes(searchString [,position])`**
- **startsWith()** – check if a string starts with another string.
 - **Syntax : `string.startsWith(searchString [,position])`**
- **endsWith()** – determine if a string ends with another string
 - **Syntax : `String.endsWith(searchString [,length])`**

JavaScript String

- **trim()** – remove whitespace characters from a string.
 - Syntax : **string.trim()**
- **trimStart()** – remove the leading whitespace characters of a string.
 - Syntax : **string.trimStart();**
- **trimEnd()** – remove the ending whitespace characters of a string.
 - Syntax : **string.trimEnd();**
- **split()** – split a string into an array of substrings.
 - Syntax : **split([separator, [,limit]]);**

JavaScript String

- **replace()** – replace a substring in a string with a new one.
 - **Syntax : str.replace(substr, newSubstr)**
- **replaceAll()** – replace all occurrences of a substring that matches a pattern with a new one.
 - **Syntax : str.replaceAll(substr, newSubstr)**
- **toUpperCase** – return a string with all characters converted to uppercase.
 - **Syntax : str.toUpperCase()**
- **toLowerCase** – return a string with all characters converted to lowercase.
 - **Syntax : str.toLowerCase()**

JavaScript Events

- An HTML event can be something the browser does, or something a user does.
- Some examples of HTML events:
 - An HTML web page has finished loading
 - An HTML input field was changed
 - An HTML button was clicked
- Often, when events happen, user may want to do something.
- JavaScript lets you execute code when events are detected.

JavaScript Events

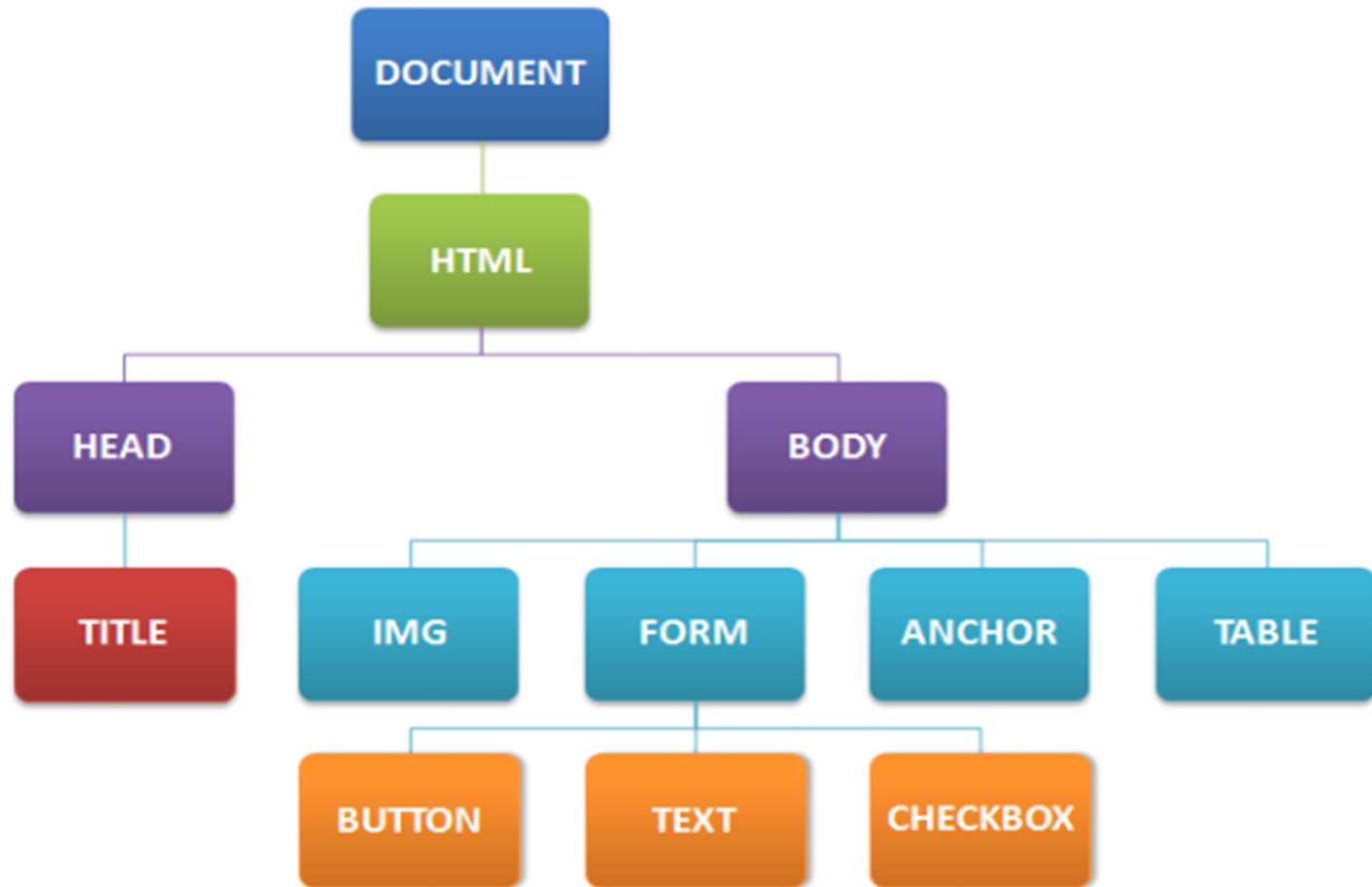
■ Common HTML Events are :

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

Document Object Model

- The Document Object Model (DOM) is an application programming interface (API) for manipulating HTML documents.
- The DOM represents an HTML document as a tree of nodes. The DOM provides functions that allow you to add, remove, and modify parts of the document effectively.
- DOM is cross-platform and language-independent ways of manipulating HTML and XML documents.

Document Object Model



Document Object Model - Selecting elements

- **getElementById()** – select an element by id.
- **getElementsByName()** – select elements by name.
- **getElementsByTagName()** – select elements by a tag name.
- **getElementsByClassName()** – select elements by one or more class names.
- **querySelector()** – method returns the first element that matches a CSS selector.
- **querySelectorAll()** : method returns all elements that matches a CSS selector(s). It returns a NodeList.

Document Object Model - Manipulating elements

- **createElement()** – create a new element.
- **textContent** – get and set the text content of a node.
- **append()** insert a node after the last child node of a parent node.
- **prepend()** – insert a node before the first child node of a parent node.
- **insertBefore()** – insert a new node before an existing node as a child node of a specified parent node.

```
parentNode.insertBefore(newNode, existingNode);
```

- **removeChild()** – remove child elements of a node.

Javascript setTimeout()

- The **setTimeout()** method executes a block of code after the specified time. The method executes the code only once.

```
// program to display a text using setTimeout method
function greet() {
    console.log('Hello world');
}

setTimeout(greet, 3000);
console.log('This message is shown first');
```

Javascript setInterval()

- The **setInterval()** method repeats a block of code at every given timing event.

```
// program to display a text using setInterval method
function greet() {
    console.log('Hello world');
}

setInterval(greet, 1000);
```

Array

Adding/removing elements

push() – add one or more elements to the end of an array.

Syntax : Array.push(item1, item2 ...)

unshift() – add one or more elements to the beginning of an array.

Syntax : Array.unshift(item1, item2 ...)

pop() – remove an element from the end of an array.

Syntax : Array.pop()

shift() – remove the first element from an array.

Syntax : array.shift()

slice() – copy elements of an array.

Syntax : slice(start, stop);

Array splice

- **Array splice: Delete, Insert, and Replace** : This method is to delete existing elements, insert new elements, and replace elements in an array.
- **Insert** : Insert one or more elements into an array by passing three or more arguments to the splice() method with the second argument is zero.

```
Array.splice(position, 0, new_element_1, new_element_2, ...);
```

Array splice

- The position specifies the starting position in the array in which the new elements will be inserted.
- The second argument is zero (0) which instructs the splice() method to not delete any array elements.
- The third argument, fourth argument, and so on are the new elements that are inserted into the array.
- Note that the splice() method changes the original array.

Array splice

- **Replacing elements** :The splice() method allows you to insert new elements into an array while deleting existing elements simultaneously.
- The number of elements to delete needs not be the same as the number of elements to insert.

```
let arr = [11,22,33,44,55];  
arr.splice(1, 1, 0);  
console.log(arr);
```

- **Output :**

```
[11, 0, 33, 44, 55]
```

Array splice

- **Deleting elements** : To delete elements in an array, you pass two arguments into the splice() method as follows:

```
Array.splice(position,num);
```

- The **position** specifies the position of the first item to delete and the **num** argument determines the number of elements to delete.
- This method changes the original array and returns an array that contains the deleted elements.

Array

- **Finding elements**
- **indexOf()** – This method returns the index of the first occurrence the element that you want to find, or -1 if the element is not found.
- **Syntax : `Array.indexOf(searchElement, fromIndex)`**
- **includes()** – check if an element is in an array.
- **Syntax : `array.includes(element,fromIndex);`**

Array forEach

forEach() : This method allows you to run a function on every element.

```
let arr = ['A', 'B', 'C'];  
arr.forEach(function (e)  
{  
    console.log(e);  
});
```

JavaScript spread operator

ES6 provides a new operator called spread operator that consists of three dots (...). The spread operator allows you to spread out elements of an iterable object such as an array, map, or set.

```
function fn()
{
    const odd = [1,3,5];
    const combined = [2,4,6, ...odd];
    console.log(combined);
}

fn(); // [ 2, 4, 6, 1, 3, 5 ]
```

JavaScript Destructuring Assignment

ES6 provides a new feature called destructuring assignment that allows you to destructure properties of an object or elements of an array into individual variables.

```
function getScores() {  
    return [70, 80, 90, 100];  
}  
  
let [x, y, z] = getScores();  
  
console.log(x); // 70  
console.log(y); // 80  
console.log(z); // 90
```

```
const marks = {  
    section1: { alpha: 15, beta: 16},  
    section2: { alpha: -31, beta: 19 }  
};  
  
const { section1 : { alpha: alpha1, beta: beta1 }} = marks;  
console.log(alpha1, beta1); // 15, 16
```

JavaScript Objects

Objects are variables too. But objects can contain many values.

The values are written as **name:value** pairs (name and value separated by a colon).

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};  
  
console.log(person['firstName']);  
console.log(person.lastName);
```

JavaScript this

- In JavaScript, the this keyword refers to an object.

The this keyword refers to different objects depending on how it is used:

- In an object method, this refers to the object.
- Alone, this refers to the global object.
- In a function, this refers to the global object.
- In an event, this refers to the element that received the event.

JavaScript this

- In JavaScript, arrow functions do not have their own this context.
- Arrow functions are designed to inherit this from the lexical scope (the surrounding context where they are defined). This means that, instead of having their own this (like regular functions), arrow functions capture this from the outer scope. This is called lexical scoping.

JavaScript Function binding

- Function binding in JavaScript happens using the Bind() method.
- This method can bind an object to a common function, so that the function gives different results when needed.
- JavaScript uses the Bind() method to call a function with 'this' value.

JavaScript constructor functions

- Program often need to create many similar objects .
- To do that, JavaScript uses constructor function to define a custom type and the new operator to create multiple objects from this type.
- Constructor function is a regular function with the following convention:
 - The name of a constructor function starts with a capital letter like Person, Document, etc.
 - A constructor function should be called only with the new operator.

JavaScript constructor functions

- The following example defines a constructor function called Person:

```
function Person(firstName, lastName) {  
    this.firstName = firstName;  
    this.lastName = lastName;  
}
```

- To create a new instance of the Person, use the **new** operator:

```
let person = new Person('John', 'Doe');
```

JavaScript prototype

- In JavaScript, objects can inherit features from one another via prototypes. Every object has its own property called a prototype.
- Because the prototype itself is also another object, the prototype has its own prototype. This creates a something called prototype chain. The prototype chain ends when a prototype has null for its own prototype.